



Kubernetes Mimarisi

Kubernetes ve Konteyner Dünyası



Bu Konuda

- Kubernetes cluster yapısı
- Control plane bileşenleri
- Node bileşenleri
- Control plane ↔ node iletişimi
- Controller'ların API üzerindeki yeri
- Garbage collection ve finalizer'lar

Kubernetes Cluster Yapısı





Kubernetes Cluster Yapısı

- Bir Kubernetes cluster'ı, bir **control plane** ve **bir veya daha fazla worker node**'tan oluşur^{[1],[2]}
- **Control plane:** cluster hakkında global kararlar alır (ör. scheduling) ve cluster olaylarını algılayıp yanıtlar (ör. eksik replica'ları başlatma)^[1]
- **Worker node'lar:** node'a yerleştirilen en küçük workload birimi olan Pod'ları barındırır^[1]
→ **Pod ve Pod Yaşam Döngüsü**
- Pod çalıştırmak için cluster'da **en az bir worker node** olmalıdır^[1]
- Gerçek ortamda control plane genellikle **birden fazla makinede** çalışır (hataya dayanıklılık (fault-tolerance) ve yüksek erişilebilirlik (HA))^[1]
- Control plane bileşenleri cluster içinde **herhangi bir makinede** çalışabilir; kurulum script'leri genellikle hepsini **aynı makinede** başlatır ve orada kullanıcı konteyneri çalıştırmaz^[1]
- Küçük veya geliştirme cluster'larında control plane bileşenleri ve iş yükleri **aynı node'lar** üzerinde çalışabilir; gerçek ortamda büyük cluster'lar genellikle control plane için **ayrı node'lar** ayırır^[1]

Control Plane Bileşenleri





kube-apiserver

- Kubernetes **API'sini dışa açan** control plane bileşenidir; control plane'in **ön yüzüdür**^[1]
- Cluster'daki **tek merkez (hub)**; `kubectl`, kubelet, scheduler ve controller'ların hepsi API server ile konuşur^[3]
- **etcd ile konuşan tek bileşen** API server'dır; tutarlılığın ve yetkilendirmenin merkezidir
- **Yatay ölçeklenebilir**: birden fazla kopya çalıştırılıp trafik dengelenebilir^[1]
- REST API, doğrulama ve admission iş akışının merkezidir
 - **Kubernetes API ve Manifest Okuryazarlığı**



etcd

- Tutarlı (consistent) ve yüksek erişilebilir bir **anahtar-değer (key-value) deposu**dur^[1]
- Kubernetes'in **tüm cluster verisinin** arka plandaki veri deposudur (backing store)^[1]
- Control plane'in kalıcı state'i burada yaşar; yedeği alınmalıdır^[1]
 - **Yüksek Erişilebilirlik ve etcd**



kube-scheduler

- **Node atanmamış yeni Pod'ları** izler ve her Pod için **uygun bir node seçer**^[1]
- Seçimde dikkate alınan etkenler:
 - Kaynak gereksinimleri, donanım/yazılım/politika kısıtları
 - Affinity ve anti-affinity kuralları
 - Veri yerelliği (data locality), iş yükleri arası etkileşim ve süreler
- Kullanıcı isterse özel scheduler'lar da kurabilir
 - **Scheduling ve Yerleşim**



kube-controller-manager

- **Controller süreçlerini çalıştıran** control plane bileşenidir^[1]
 - Mantıksal olarak her controller ayrı bir süreçtir; ancak karmaşıklığı azaltmak için **tek bir binary'de** toplanırlar^[1]
 - Yerleşik controller örnekleri^[1]:
 - **Node controller:** node'ların çöküşünü algılar ve yanıtlar
 - **Job controller:** tek seferlik işleri izler ve bu işleri çalıştıracak Pod'ları oluşturur
 - **EndpointSlice controller:** Service ile Pod'lar arası bağlantıyı kurar
 - **ServiceAccount controller:** yeni namespace'ler için varsayılan ServiceAccount oluşturur
- **Workload'lar ve Controller'lar**



cloud-controller-manager

- **Buluta özgü kontrol mantığını** içine gömen control plane bileşenidir; cluster'ı **bulut sağlayıcı API'sine** bağlar^[1]
- Kubernetes ile alttaki bulut altyapısı arasındaki **birlikte çalışabilirlik (interop) mantığını** ayırır^[1]
- **Yalnızca bulut sağlayıcıya özgü controller'ları** çalıştırır:
 - **Node controller:** node silinmiş mi diye buluttan kontrol eder
 - **Route controller:** alttaki bulut ağında yönlendirmeleri kurar
 - **Service controller:** bulut sağlayıcı load balancer'larını oluşturur/günceller/siler
- Kendi altyapınızdaki (on-premises) veya öğrenme ortamlarındaki cluster'larda bulunmaz^[1]
→ **Ekosistem ve Servis Sağlayıcıları**

Node Bileşenleri





Node Bileşenleri

- Node bileşenleri **her node'da** çalışır; çalışan Pod'ları sürdürür ve Kubernetes runtime ortamını sağlar^[1]

Bileşen	Rolü
kubelet	Node üzerinde Pod/konteynerleri yönetir
kube-proxy	Service için ağ kurallarını uygular (ağ eklentisi kendi proxy'sini sağlıyorsa gerekmez)
container runtime	Konteynerleri çalıştırır



kubelet

- Her node'da çalışan **bir agent**; konteynerlerin bir Pod içinde **çalıştığından emin olur**^[1]
- Çeşitli mekanizmalarla sağlanan **PodSpec'leri** alır ve bunlarda tanımlı konteynerlerin **çalıştığından ve sağlıklı olduğundan** emin olur^[1]
- Kubernetes tarafından **oluşturulmamış konteynerleri yönetmez**^[1]
 - **Cluster Kurulumu ve Node İşlemleri**



kube-proxy

- Her node'da çalışan bir **network proxy**'sidir; Kubernetes **Service** kavramının bir parçasını uygular^[1]
- Node'lar üzerinde **ağ kuralları** tutar; bu kurallar Pod'lara içeriden veya dışarıdan erişimi sağlar^[1]
- İşletim sisteminin paket filtreleme katmanını kullanır; yoksa trafiği **kendisi iletir**^[1]
- Ağ eklentisi kendi proxy'sini sağlıyorsa **gerekmez**^[1]
 - **Ağ Modeli ve Uygulama Erişimi**



Container Runtime ve CRI

- Konteynerlerin **çalıştırılmasından sorumlu** yazılım; her node'da bulunur (containerd, CRI-O) ^[1]
- Kubernetes, runtimes ile **CRI (Container Runtime Interface)** üzerinden konuşur ^[6]
 - **gRPC tabanlı eklenti arayüzü** → kubelet, runtime'ları yeniden derlemeden kullanabilir
 - Kubelet **istemci**, runtime **sunucu** konumundadır
 - **v1** CRI API: v1.23'te stabil, v1.26+ kubelet için zorunlu (v1 desteklemeyen node kayıt olmaz)
→ **Cluster Kurulumu ve Node İşlemleri**

Control Plane–Node İletişimi





Node'dan Control Plane'e İletişim

- Kubernetes bir "**hub-and-spoke**" **API deseni** kullanır^[3]:
 - Node'lardan (ve Pod'lardan) gelen **tüm API kullanımı API server'da sonlanır**
 - Diğer control plane bileşenleri **uzak servis açmaz**
- Node'dan control plane'e bağlantı, güvenli **HTTPS** portu (genellikle 443) üzerinden **kimlik doğrulama ve yetkilendirme** ile yapılır^[3]
- Node'lar, cluster'ın **kök sertifikası** ve geçerli istemci kimlik bilgileriyle sağlanır^[3]
- Control plane bileşenleri de API server ile güvenli **HTTPS** portu üzerinden konuşur^[3]
- Varsayılan mod **güvenlidir**; güvenilmeyen veya ortak ağlarda çalıştırılabilir^[3]
 - **Kimlik Doğrulama, Yetkilendirme ve RBAC**



Control Plane'den Node'a İletişim

API server, node'lara yalnızca belirli kullanıcı işlemleri için ulaşır; iki temel yol vardır^[3]:

Yol	Kullanım
API server → kubelet	Pod log'ları, attach ve port-forward (<code>kubectl logs</code> , <code>kubectl exec</code> ...)
API server → node/pod/servis	API server'ın proxy işlevi

- API server → kubelet bağlantısı, kubelet'in HTTPS uç noktasında sonlanır; varsayılan olarak kubelet sertifikası **doğrulanmaz** (ortak ağlarda güvensiz) ^[3]
- API server → node/pod/servis bağlantıları varsayılan olarak **plain HTTP**'dir (şifrelenmemiş ve kimlik doğrulamasız) ^[3]
- SSH tünelleri deprecate edilmiştir; yerine **Konnectivity** servisi kullanılır ^[3]
- Controller'lar node'lara doğrudan bağlanmaz; yalnızca API server üzerinden çalışır. Node tarafındaki kubelet, kendi node'una atanan Pod'ları API server'ı izleyerek çeker

Controller'ların API Üzerindeki Yeri





Controller'ın API Üzerindeki Rolü

- Controller'lar genellikle işi kendileri yapmaz; **API server'a yararlı yan etkileri olan mesajlar** gönderir^[4]
- Örnek: Job controller yeni bir Job görünce, işi bitirecek Pod'ları oluşturması için API server'a istek yapar^[4]
- Aynı türdeki nesneyi **birden çok controller** üretebilir; örn. Job ve Deployment'ın controller'larının ikisi de Pod oluşturur ve onları **label** ile ayırt eder^[4]
- Bazı controller'lar cluster **dışındaki** durumla doğrudan etkileşir (örn. node sayısını ölçekleyen controller)^[4]
→ **Workload'lar ve Controller'lar**



Garbage Collection

- Kubernetes, bazı kaynakları otomatik temizler: sonlanan Pod'lar, tamamlanan Job'lar, sahibi (owner) olmayan nesneler vb. ^[5]
- **Owner/dependent ilişkisi:** nesneler `ownerReference` ile birbirine bağlanır; GC, bir nesne silinirken bağımlılarını temizleyebilir ^[5]
 - `Kubernetes API ve Manifest Okuryazarlığı`
 - Owner ilişkisi **label/selector**'dan farklıdır ^[5]
- **Cascading deletion:** foreground (önce bağımlılar, sonra owner) ve background (owner önce, bağımlılar arkada) ^[5]
- **Finalizer:** nesnenin silinmeden önce temizlik işlerinin tamamlanmasını sağlar ^[5]
- kubelet, kullanılmayan imajları **5 dakikada bir**, kullanılmayan konteynerleri **1 dakikada bir** temizler ^[5]
 - `Depolama`, `Workload'lar ve Controller'lar`

Kaynaklar





Kaynaklar

1. Kubernetes Mimarisi: kubernetes.io/docs/concepts/architecture/
2. Kubernetes Bileşenleri: kubernetes.io/docs/concepts/overview/components/
3. Control Plane-Node İletişimi: kubernetes.io/docs/concepts/architecture/control-plane-node-communication/
4. Controllers: kubernetes.io/docs/concepts/architecture/controller/
5. Garbage Collection: kubernetes.io/docs/concepts/architecture/garbage-collection/
6. CRI: kubernetes.io/docs/concepts/containers/cri/